
Native T-SQL Language Service

Pure Analysis, Pinned Metadata, Feature Routing, and SQL Scripting

Design, implementation, and validation review

For [vscode-mssql PR #22860](#)

Central question. How can `vscode-mssql` replace remote, document-coupled language features with a deterministic TypeScript engine that remains useful on incomplete SQL, pins one immutable metadata generation per answer, distinguishes syntax truth from catalog-dependent claims, and refuses to turn uncertainty into false diagnostics or authoritative scripts?

Assessment at the reviewed head. PR #22860 establishes a coherent, source-only language and scripting foundation. Its dependency boundaries, pinned metadata seam, deterministic feature pipeline, stale-safe diagnostics scheduler, and strict scripting provenance are sound. Review commit `b66db2d8f` closes the original 26 threads and one immediate follow-up, including the eight reproduced ordinary-SQL trust defects, with focused regression coverage and a real catalog-builder/store/provider-to-engine path. Exact-head local validation passed 707 owned language/scripting tests, 47 adjacent tests, the 4,742-test broad MSSQL lane, SQL Projects, build, lint, formatting, hooks, and online packaging; all hosted checks are green and the packaged MSSQL VSIX has a zero-byte delta. A later review found one residual spaced alias-column-list variant, so automated closeout has one valid correctness thread remaining; consumer activation remains gated independently on preview composition and integration proof.

Prepared for:	Karl Burtram
Primary change:	<code>dev/karlb/query_05_language_service</code> at <code>b66db2d8f</code>
Extraction base:	<code>95cf790e5</code> ; 59 files, +26,391 / -0
Live PR base:	<code>53b707bbe</code> on <code>main</code>
Current <code>main</code> :	<code>c6d2baa442</code> ; topic ahead 3 / behind 9
Metadata prerequisite:	merged PR #22836 / shared metadata substrate
Focused corpus:	707 owned / 754 total focused tests
Report snapshot:	31 August 2026, America/Los_Angeles

This is an as-built, evidence-qualified report. Historical design notes explain intent; the pinned implementation, exact-head tests, live review state, Actions state, and package artifacts control where the documents and implementation differ.

Contents

How to read this report	1
1 Review snapshot and evidence model	1
2 Executive summary	3
3 Placement in the refactor train	4
4 Public contracts and dependency boundaries	5
5 Document analysis pipeline	6
6 Metadata contract and honest degradation	8
7 Language feature behavior	10
8 Scheduling, routing, and failure containment	11
9 Definition and SQL scripting	13
10 Observability and privacy	14
11 Test design and validation	15
12 Known limitations and review closure	18
13 Integration and graduation plan	21
14 Final assessment	23
Evidence and source map	24

How to read this report

The report keeps implementation, intended invariants, executable evidence, and future integration separate:

Implemented

Present in the pinned PR head and traced to source.

Target invariant

A behavior or safety contract the library is intended to preserve.

Validated Exercised by a named test, build lane, package check, or observed repository state.

Open finding

A live review finding or evidence gap that remains at report freeze.

Reported evidence

A source-attributed measurement or probe not independently rerun while preparing this document.

Future contract

A consumer, bridge, activation path, or maturity step not shipped by this PR.

Key takeaway

PR #22860 is not a replacement language service that users can turn on. It is the independently reviewable library beneath such a replacement: public data contracts, a tolerant analysis pipeline, native features, a pinned metadata adapter, strict scripting, routing/scheduling policies, observability, and tests. A later PR must supply a gated consumer and VS Code adaptation before any product behavior changes.

Evidence boundary

The strongest evidence is deterministic unit behavior and source-boundary inspection. Local packaging proves the tree packages; GitHub's package comparison more directly proves that the new source contributes zero size to the shipped VSIX at this head. Neither proves live-catalog semantic equivalence across the full T-SQL grammar. That is a later preview-graduation gate.

1 Review snapshot and evidence model

1.1 Pinned repository state

The review separates three different baselines that are easy to blur together. The feature was authored on extraction base 95cf790e5; the pull request currently identifies 53b707bbe as its live base; and the repository's `main` branch has since advanced to c6d2baa442. The exact reviewed topic is three commits ahead of and nine commits behind current `main`. This does not invalidate the isolated source review, but a future consumer must begin from refreshed `main`; a maintainer may also choose to refresh this inert extraction before merge.

Table 1: Current-head repository and evidence snapshot.

Dimension	Observed value	What it proves
PR head	b66db2d8fde1	Exact source and local validation target reviewed in this report.
Extraction base	95cf790e5cfe	Metadata PR #22836 is already present in the authored topic history.
Live PR base	53b707bbed5d	Base recorded by the active pull request.
Current main	c6d2baa44247	Nine mainline-only commits remain outside the isolated topic.
Change size	59 files; +26,391 / -0	A large but isolated source extraction, dominated by new language, scripting, and test files.
GitHub workflows	Build/test, CodeQL, normalization, and six-platform VSIX launch all green	Existing build, lint, hosted test, package, and launch lanes pass at the pinned head.
Package comparison	MSSQL 85,347 KiB on target and PR; 0 KiB delta	The new source is not an input to the shipped extension bundle.
Coverage	94.34331% patch at the reported run; 1,041 changed lines uncovered	Broad execution, but not semantic completeness; coverage is supporting evidence rather than the trust boundary.
Review state	27 resolved threads; one later residual thread open	Original closeout is complete; the ordinary spaced alias-list variant is a valid final correctness repair.

1.2 Evidence ladder

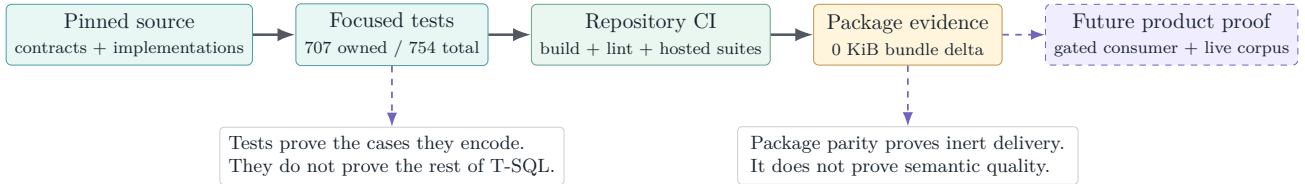


Figure 1: Evidence ladder. Each layer strengthens a different claim; green CI and zero package delta cannot substitute for a live semantic truth corpus.

Evidence boundary

The strongest current claims are architectural isolation, deterministic behavior on the focused corpus, package invisibility, and repair coverage for the original review findings. The weakest claim remains real-world semantic authority across the full T-SQL grammar. The external differential review improved the honesty corpus and then found one residual whitespace variant; that is exactly why activation requires a broader live differential corpus rather than relying on aggregate coverage alone.

1.3 Executive scorecard

Table 2: Design objective versus current evidence.

Objective	State	Assessment
Pure, consumer-neutral engine boundary	✓	JSON-safe, VS Code-neutral contracts and linted import direction.
One metadata generation per answer	✓	Every feature pins an immutable view; definition is the single sanctioned lazy read.
No metadata wait on keystroke paths	✓	Interactive features read synchronously and may only request background hydration.
Honest diagnostic suppression	△	The original eight reproduced legal-SQL defects are repaired; one spaced alias-column-list variant remains open.
Stale-result containment	✓	Scheduled diagnostics stamp document version plus metadata generation and abort stale publication.
Per-feature fallback containment	△	Throw/rejection circuits work; a timer cannot preempt synchronous extension-host CPU work.
Strict scripting provenance	✓	Freshness/refusal/offline provenance and pathological comment sanitization are explicit and regression-tested.
Production metadata-adapter confidence	△	A real builder/snapshot/provider-to-engine test now exists; broader live-server and generation-drift coverage remains preview work.
Private-preview invisibility	✓	No activation, provider registration, command, setting, or bundle input lands in this PR.
Current-main merge readiness	△	The branch is mergeable and fully green, but one valid review repair remains and the topic is nine mainline commits behind.

2 Executive summary

2.1 What the PR establishes

The change adds 37 production files under `src/sqlLanguage`, six under `src/sqlScripting`, fifteen focused unit files, and one observability classification edit. The architecture has four load-bearing properties:

1. **Pure request contracts and analysis.** Positions are zero-based UTF-16; payloads are JSON-safe; the core has no VS Code document or provider types.
2. **One truth generation per request.** Language work pins one immutable metadata view and reads readiness, environment, names, columns, parameters, keys, and definitions through that generation-stable seam.
3. **Honest partial behavior.** Absence and unavailability are different. The engine suppresses metadata-dependent claims, returns incomplete completion where appropriate, and preserves lexical/structural diagnostics when catalog validation is unavailable.
4. **Source-only private-preview boundary.** There is no package contribution, command, setting, activation event, controller, provider registration, Query Studio bridge, or AI consumer. Existing users cannot reach the code from this PR.

2.2 What the PR does not establish

It does not claim full T-SQL parsing, ScriptDOM equivalence, semantic-token or document-highlight support, live runtime integration, a production latency service-level objective, or a user-visible migration

from the current SQL Tools Service language providers. The router includes a lazy bridge seam, but the bridge itself and the host composition that chooses it are deferred.

Private-preview meaning in this PR

No dedicated setting is added because there is no activated feature to gate. That is safer than a dead setting that suggests usable behavior. The first integration PR must require the umbrella experimental/private-preview flag *and* a dedicated feature-area flag, default both off, hide all contributed UI, and register runtime consumers only when the effective gate is true at activation.

2.3 The claim hierarchy

The implementation is best understood as a ladder of increasingly expensive claims. Each higher level depends on every lower level plus additional authority. When a lower level is uncertain, the engine should retain lower-level usefulness and remove the unsupported higher-level claim.

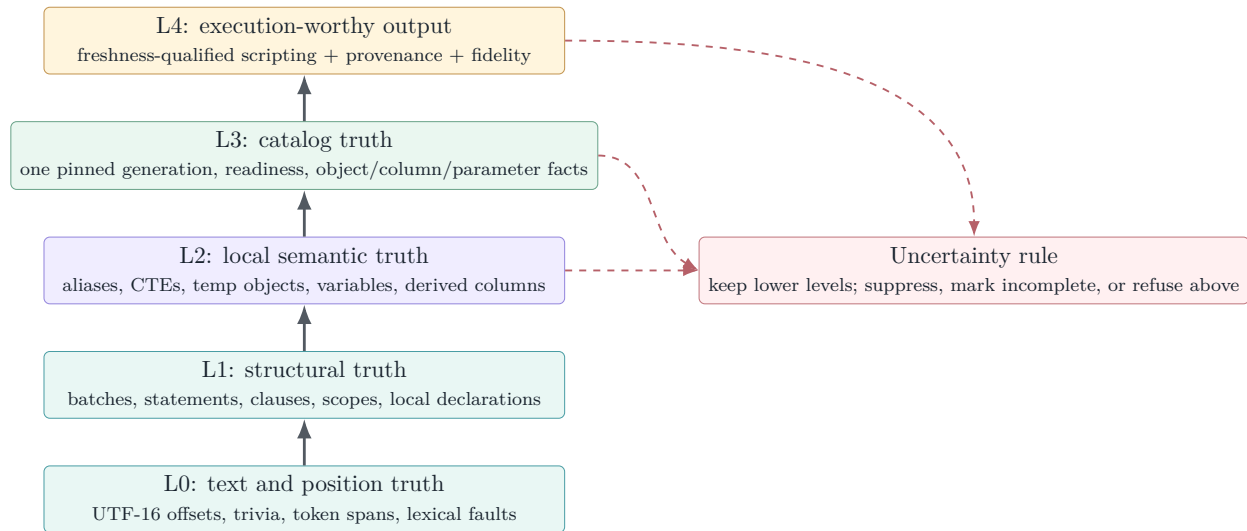


Figure 2: Claim hierarchy. The engine is useful before it is omniscient: uncertainty removes authority rather than fabricating certainty.

The central safety invariant

A missing token may reduce structural confidence; a missing metadata section may reduce catalog confidence; an unvalidated snapshot may block scripting. None of those conditions should erase facts from a lower tier that remain independently true. This is why lexical diagnostics can continue while binder diagnostics are suppressed, and why keywords/local symbols can remain available while catalog completion is incomplete.

3 Placement in the refactor train

PR #22860 is extraction PR05. The accepted refactor layers beneath it are already in the authored base: core observability, SQL Data Plane and STS2 transport, and the shared metadata substrate from PR #22836. This PR adds the pure language/scripting library and the production metadata adapter, but deliberately stops before any VS Code or Query Studio consumer registration.

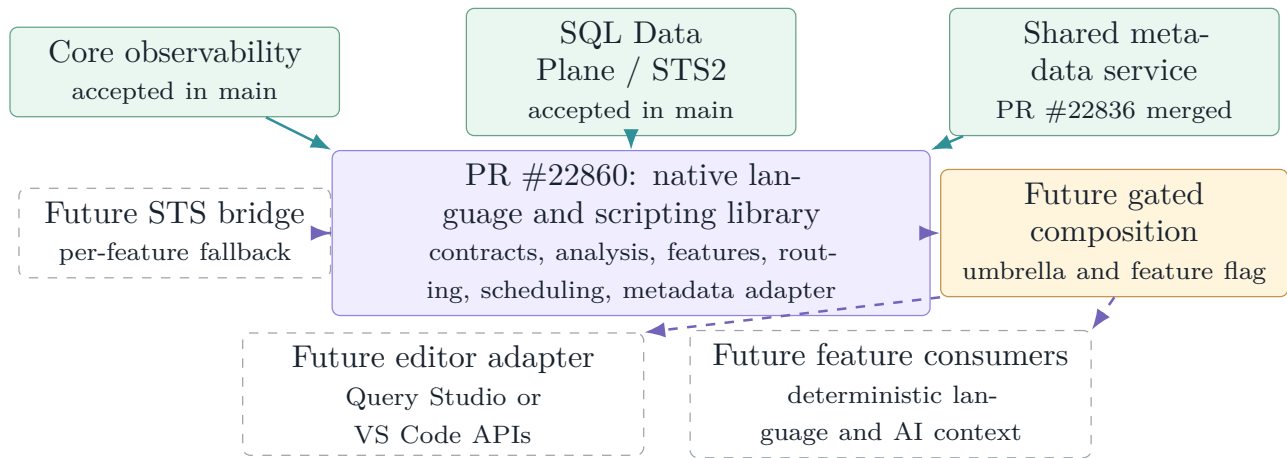


Figure 3: Refactor placement. Solid lower layers and the violet PR05 library exist; dashed consumer and fallback paths are intentionally absent from this pull request.

The package bot reports the MSSQL VSIX as 85,347 KiB on both target and PR branches, and the other packaged extensions also show zero delta. That is stronger scope evidence than merely observing that no command or setting was added: the build graph itself does not include the new source at this head.

Branch-drift note

The PR was authored from extraction base 95cf790e5; the active PR now names 53b707bbe as its base, while current `main` is c6d2baa442. The nine-mainline-commit gap touches integration surfaces such as `mainController.ts`, `sqlDocumentService.ts`, package metadata, and general extension infrastructure. The language source remains isolated and exact-head packaging is green, but every later consumer must start from refreshed `main` and rerun the integration assertions.

4 Public contracts and dependency boundaries

4.1 Consumer-neutral API

`sqlLanguage/api.ts` defines JSON-safe request and response data for completion, hover, signature help, diagnostics, definition, folding, document symbols, highlights, and semantic tokens. Point requests carry the full document text, a version, and a zero-based UTF-16 position; diagnostics/structure requests carry text and version. The contract deliberately carries no VS Code document, URI, cancellation token, metadata object, or STS DTO. A future host owns document identity, routing, cancellation, and translation to editor-native result types.

The v1 request shape sends the full document text on each call. That is a simple and highly testable extraction boundary, not a commitment to permanent full-text transport. A later Query Studio or VS Code adapter can add incremental synchronization or per-document engine instances without changing the pure feature algorithms.

The same file defines feature maturity and a capability table. The shipped native capability table marks completion, hover, signature, diagnostics, definition, folding, and document symbols as preview; highlights and semantic tokens remain off. Having types for a feature is therefore not the same as advertising an implementation.

Table 3: Public request-shape boundary.

Request family	Carries	Intentionally owned by the future host
Point features	Full text, version, UTF-16 line/character; completion also carries trigger kind/character	URI/document identity, connection binding, cancellation, VS Code/Monaco result adaptation.
Diagnostics/structure	Full text, version; diagnostics may carry a host-derived metadata freshness verdict	Debounce, scheduling, publication, editor marker ownership, current metadata stamp.
Definition/scripting	Point request plus target resolution; scripting request plus provenance/fidelity contracts	Virtual document storage, reveal behavior, strict lease policy, user-facing refusal UX.

Table 4: Native feature surface at the pinned head.

Feature	Capability	As-built behavior
Completion	Preview	Context/expectation-gated candidates, deterministic rank, partial-readiness honesty, lazy hydration kick.
Hover	Preview	Local, overlay, catalog, system, routine, built-in, and description facts when their readiness tier permits.
Signature help	Preview	Built-ins, catalog functions/procedures, nested calls, and EXEC argument tracking.
Diagnostics	Preview	T1 lexical/structural errors plus freshness-gated T2 208/207/209 binder warnings.
Definition	Preview	In-document local targets or virtual scripted catalog definitions with anchors and fidelity.
Folding	Preview	Batch, statement, comment, and region ranges.
Document symbols	Preview	Batch/statement structure and simple create/alter object declarations.
Highlights	Off	Interface reserved; native engine returns no result.
Semantic tokens	Off	Interface reserved; native engine returns no result.

4.2 Purity and import direction

Core analysis, feature computation, static data, provider interfaces, and scripting emitters are library code. Concrete host concerns live under `host`: metadata freshness, diagnostic journaling, timeout/yield behavior, and routing. The one sanctioned bridge from language metadata to the product catalog model is `provider/catalogProvider.ts`.

Design invariant

Interactive feature code consumes a synchronous `IPinnedMetadataView`; it does not hold a mutable store, lease, network client, VS Code document, or STS service. Definition text is the one deliberate async lookup on the pinned view, and strict scripting asks the host to establish freshness before it creates the engine.

5 Document analysis pipeline

5.1 From text to reusable structure

The native engine analyzes one immutable text snapshot, then reuses structural work across feature calls. Review commit `b66db2d8f` strengthens both analysis and diagnostics memoization to require exact text identity in addition to version/length context. Same-version, same-length replacement text can no longer reuse the wrong analysis, while ordinary repeat requests still share immutable structural work.

The stages are deliberately tolerant:

1. **Text snapshot** maps offsets to zero-based UTF-16 line/character positions and back, including ordinary LF/CRLF and mixed input.
2. **Lexer** recognizes T-SQL words, identifiers, punctuation, comments, strings, variables, SQLCMD-sensitive regions, Unicode horizontal whitespace, and `GO` / `GO n` line semantics. Review repairs pin non-breaking-space classification and a construct opener at exact EOF.
3. **Segmenter** partitions batches and statements while respecting module bodies and rule-sensitive constructs.
4. **Sketch parser** extracts the structure features need: query scopes, sources, CTEs, select items, declarations, `EXEC`, DML targets, and table declarations. It remains total on incomplete editor text.
5. **Overlay** models script-local state such as temporary tables, declared table variables, scripted tables, and batch-scoped variables.
6. **Binder and cursor expectation** combine local structure, the effective database, and one pinned metadata view for a particular feature request.

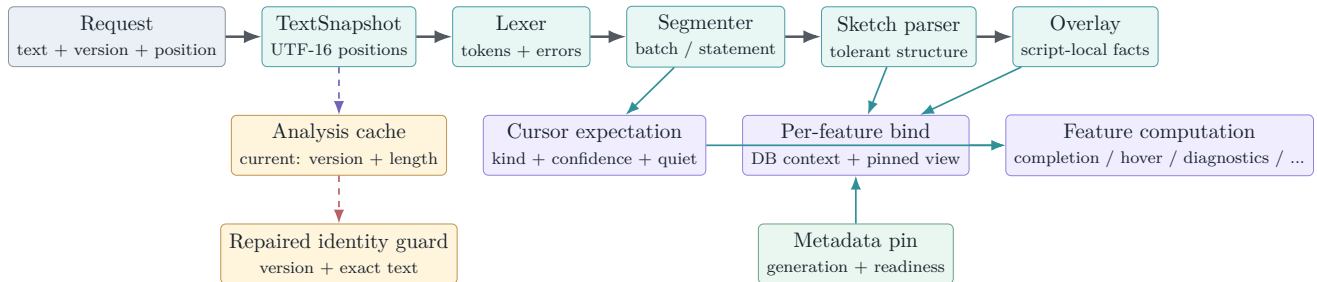


Figure 4: As-built analysis pipeline. Structural analysis is reused, metadata is pinned per request, and binding never mixes generations. The red review boundary is the analysis memo key: target identity should include document or content identity, not only version and length.

5.2 One request, one pin, one answer

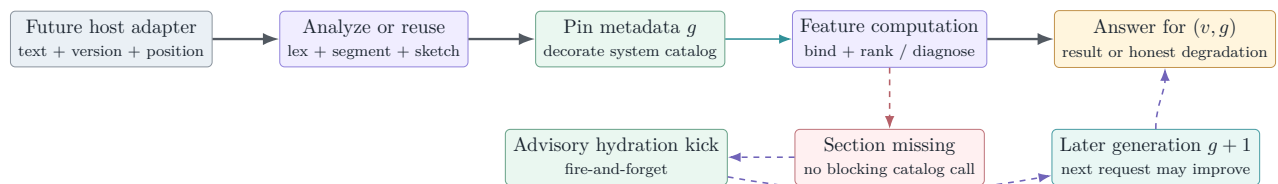


Figure 5: Interactive request lifecycle. A response is stamped by document version and one metadata generation; missing data can improve a later request but cannot mutate the answer already being produced.

5.3 Why a sketch instead of a full AST

Editor input is commonly unfinished. A full grammar can make a single missing token collapse useful context; an unbounded hand-written parser can accidentally become a second compiler. The sketch deliberately extracts only what current features consume and records uncertainty. The cursor-expectation layer improves completion category selection and suppresses suggestions in declarations, data-type positions, comments, strings, and other quiet contexts, but it is not the richer parser-v2/AST envisioned by later design notes.

Semantic ceiling

The tolerant pipeline is broad, not complete. Dynamic SQL, unusual procedural forms, deeply dialect-specific syntax, and ambiguous recovery can exceed the sketch. The correct failure mode is suppression, incomplete results, or fallback—never a confident catalog error manufactured from an untrusted parse.

6 Metadata contract and honest degradation

6.1 Generation-stable provider seam

`provider/types.ts` separates a mutable provider from the immutable view used by a request. The provider exposes current generation, environment, readiness, a pin operation, databases, an explicit hydration request, and a change event. The pin exposes synchronous resolution and projection over an opaque `LangObjectRef`; consumers cannot retain mutable catalog objects or invent cross-generation identity.

Readiness is sectioned across objects, columns, parameters, foreign keys, and definitions. The global mode—full, lite, partial, or offline—does not erase section truth. Resolution is a tagged result: resolved, default-schema choice, ambiguous, not found, or unavailable. This prevents the dangerous equation “not returned = does not exist.”

6.2 Adapter over the shared metadata service

`catalogProvider.ts` is the only production adapter from the merged metadata substrate to language types. It maps a compact immutable `CatalogSnapshot` into the pinned language surface, carries database/case/capability context, and returns honest unknown states when no snapshot exists.

When a feature discovers that object names are ready but columns are not, the adapter may request hydration. The current implementation coalesces to one in-flight refresh, avoids starting while already loading/stale, makes at most one kick per generation, and enforces a 30-second floor. The present metadata service refreshes the full ladder; the language seam is already shaped so later section-level hydration does not change feature contracts.

Table 5: Metadata-substrate to language-seam translation.

Catalog fact	Language fact	Meaning / information loss
ready	ready	Positive and negative claims for that section are permitted.
lite	partial	Some positive facts exist, but completeness must not be inferred.
loading	loading	Current pin is unchanged; feature may return incomplete and request hydration.
stale	stale	Catalog service is refreshing over an existing snapshot; not proof that data is age-stale.
failed	failed	Failure is explicit and must never be rendered as a trustworthy empty set.
absent / no snapshot	unknown	The seam cannot claim existence or non-existence.
Module definition API	definitions=lazy	One sanctioned async read, cached per metadata generation.
Snapshot mode	full / lite / partial	Structural completeness only; strict freshness and disk/offline provenance come from host policy, not this field.

The adapter is intentionally lossy

The pure engine receives a smaller truth vocabulary than the metadata store. In particular, `env.caseSensitive` is a required boolean, so the production adapter currently falls back to `false` when H0 has not established the catalog rule. Also, an offline disk snapshot retains its structural snapshot mode; ordinary language readiness is not a strict freshness verdict. Diagnostics and scripting therefore need separate host-supplied freshness data when absence claims or generated DDL require stronger authority.

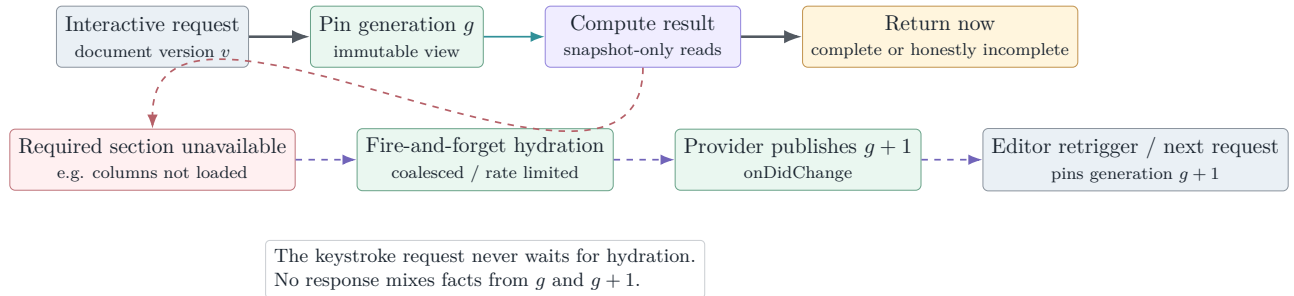


Figure 6: Lazy hydration without hot-path I/O. The current request finishes from generation g ; a later request observes $g + 1$.

6.3 System and offline views

The null provider supplies explicit offline readiness and no false catalog facts. Fixture providers simulate full, partial, and lazy-hydration states for deterministic tests. The system-catalog decorator supplies a curated names-only fallback for `sys` and `INFORMATION_SCHEMA`; live catalog facts win, identifiers are synthetic and generation-local, and the fallback does not claim column types, nullability, keys, definitions, or other facts it does not carry.

No network on the feature path

Completion, hover, signature help, and ordinary diagnostics read the pin synchronously. A hydration request is advisory and asynchronous. Strict scripting is different by design: its host establishes an explicit freshness verdict before constructing the pinned engine. The separation makes latency and truth policy visible rather than allowing an accidental catalog call inside fuzzy matching or binding.

7 Language feature behavior

7.1 Completion

Completion first classifies cursor expectation, then uses the legacy context classifier and bound statement facts to choose producers. Producers cover statement keywords/defaults/snippets, member access, table sources, joins and foreign-key predicates, expressions and columns, variables, built-ins, star expansion, INSERT, UPDATE, EXEC, DECLARE, USE, data types, and databases.

Candidate selection is deterministic: category gates run before fuzzy matching; scores are stable; ordinal comparison breaks ties; output is capped at 600. Partial metadata does not produce an authoritative empty list. Member-access on an object with unloaded columns returns an incomplete reason and can kick background hydration.

The original differential review reproduced a statement-location fallback that bypassed comment suppression plus derived-alias, UPDATE TOP, db..table, temp-overlay, case-sensitive dedupe, and bare-alias. edit gaps. Review commit b66db2d8f repairs and regression-tests those cases. A later probe found that whitespace between a derived alias and its column list still bypasses the balance skip; that remaining variant is tracked explicitly in [section 12.3](#) and is not hidden behind the word “preview.”

7.2 Diagnostics

Diagnostics intentionally separates two truth tiers:

- **T1 lexical/structural diagnostics** come from the text, tokens, segments, and tolerant statement structure. They include unterminated strings/comments and identifiers, malformed GO, bracket/parenthesis/statement recovery, and selected syntax-error 102 cases.
- **T2 binder diagnostics** depend on catalog trust: invalid object 208, invalid column 207, ambiguous column 209, and routine lookup claims such as 2812 where the sketch supports them. A host freshness verdict of `notValidated`, partial readiness, dynamic/unsupported syntax, uncertain database context, or another recorded suppression reason prevents those warnings.

The diagnostic result records suppression counts/reasons so “quiet because valid” can be distinguished from “quiet because the engine lacked authority.” Whole-document work is resumable by statement/work unit for scheduler time slicing.

The 62-case honesty suite validates the constructs it contains, not all legal T-SQL. [Section 12.3](#) records newly reproduced false positives for alias assignment, cursor syntax, WITHIN GROUP, AT TIME ZONE, multi-column derived aliases, pasted NBSP, UPDATE TOP, and default-schema multipart names. These are corpus omissions and implementation defects, not evidence against the T1/T2 separation itself.

7.3 Hover and signature help

Hover resolves schema/database/name-chain parts, aliases, projected aliases, CTEs, derived tables, temporary/script tables, table variables, variables and parameters, catalog routines, and built-ins. H7 descriptions appear only when description readiness is true. Cross-database and USE handling share the database-context map used by binding.

Signature help scans the enclosing call without requiring a complete parse. It understands nested and grouped built-in calls, catalog functions/procedures, and EXEC argument positions. Both features stay quiet in comments, strings, SQLCMD regions, and positions where the scanner cannot make an honest call.

7.4 Definition, folding, and symbols

Definition has two destinations. Local aliases, variables, CTEs, temporary objects, table variables, and derived columns return in-document ranges. Catalog objects or columns route through the scripting service and return virtual content plus semantic anchors. Folding and document symbols use only the analysis snapshot and do not require catalog I/O.

Table 6: Feature truth dependencies and degradation.

Feature	Primary truth input	When truth is incomplete
Completion	Cursor expectation, sketch, overlay, objects/columns/keys	Limit categories, return incomplete, request hydration; retain static/built-in/local candidates.
Diagnostics	Tokens/structure; then bound objects/columns	Preserve T1; suppress affected T2 and account for the reason.
Hover	Local structure or specific catalog sections	Return only facts whose section is ready; no invented type/detail.
Signature	Call scanner, built-ins, routine parameters	Built-ins/local syntax remain; catalog signatures require parameter readiness.
Definition	Local declarations or scriptable catalog target	Local range when known; virtual script with provenance; unavailable/refusal when definition truth is insufficient.
Folding/symbols	Snapshot, tokens, segments, sketch	Continue structurally; no metadata dependency.

8 Scheduling, routing, and failure containment

8.1 Sliced diagnostics scheduler

`host/scheduler.ts` is VS Code-free. A document or metadata change restarts a default 300 ms debounce. The pass then performs work in default 8 ms slices and yields by macrotask between slices. Its stamp normally combines document version and metadata generation; the scheduler rechecks the stamp before starting, after an asynchronous pass factory returns, between slices, and before publishing.

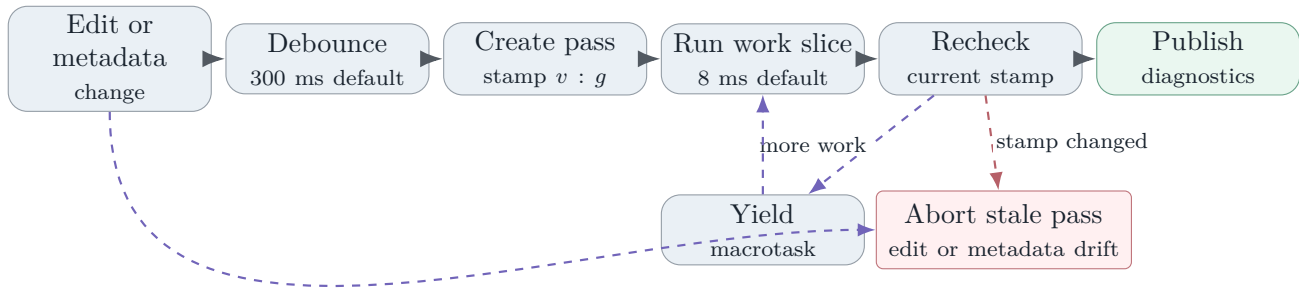


Figure 7: Diagnostics scheduling. Old results are abandoned instead of being published after either document edits or metadata-generation drift.

8.2 Two execution lanes

The implementation has two materially different scheduling models. Pull features such as completion and hover execute directly in the caller’s turn and are fast only because their algorithms and data access are bounded. Whole-document diagnostics can instead use a resumable pass whose work is sliced and yielded. The distinction matters because an ordinary JavaScript timeout observes control only after the event loop yields.

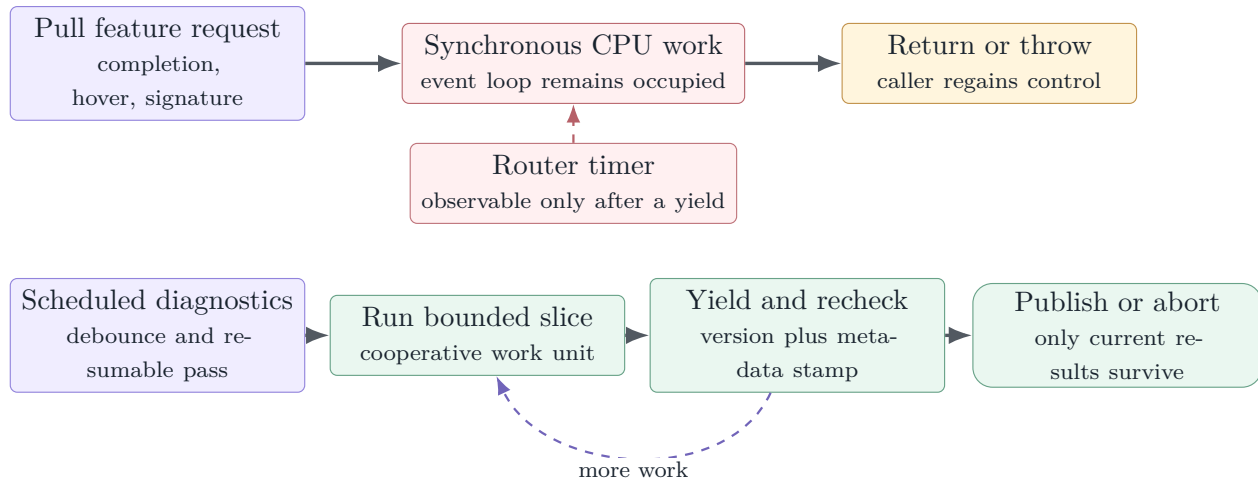


Figure 8: Execution split. Cooperative slicing bounds scheduled diagnostics; a `Promise.race` alone cannot interrupt synchronous extension-host work.

8.3 Feature router

`host/router.ts` centralizes rollout and fallback. A consumer supplies an engine preference, capability table, rollout threshold, lazy bridge factory, and optional diagnostics observer. Native work is eligible only when preference and maturity allow it. After two consecutive thrown/rejected failures for a feature, its circuit opens and later calls go directly to fallback until reset. Failures are isolated per feature, so diagnostics cannot disable completion.

The default capability table marks seven features preview, but preference still matters: under

`sqlToolsService`, only folding and document symbols remain native because the planned bridge has no equivalent path. The bridge itself is not included in this extraction.

The router wraps native work in a default five-second `Promise.race`. This is a useful containment wrapper for genuinely asynchronous operations, including definition's lazy module read, but it is not CPU preemption. Completion, hover, signature, pull diagnostics, folding, and symbols compute synchronously before their already-resolved promise can race the timer. The current tests cover preference/maturity and throw-driven circuit/reset behavior; they do not demonstrate timeout interruption. Production containment therefore requires bounded algorithms, cooperative work units, or a worker boundary for any future heavy path.

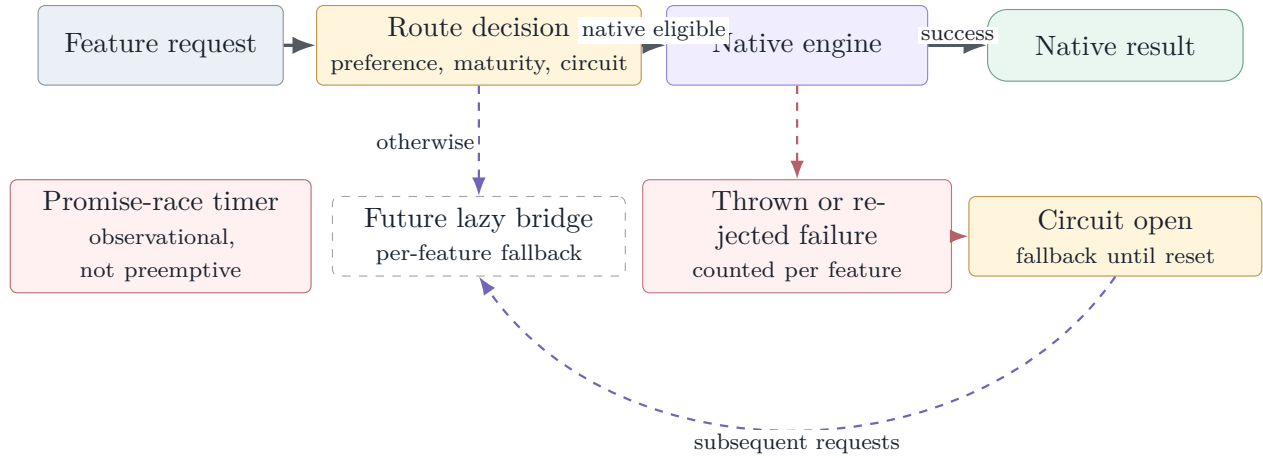


Figure 9: Router containment. Maturity and per-feature circuits are implemented; the timeout is an asynchronous observation mechanism, not an interrupt for synchronous parser or binder work.

9 Definition and SQL scripting

9.1 Scripting contract

`sqlScripting/api.ts` defines create, alter, create-or-alter, drop, drop-and-create, select-top, insert, update, delete, and execute operations. Results carry script text, anchors, fidelity, notes, unavailability reason, and metadata provenance.

Fidelity is explicit:

- **F2** is definition-level output: a stored module definition or a table statement with the required detailed metadata.
- **F1** is structurally useful but incomplete output, such as a table definition degraded by missing optional details.
- **F0** is a template, notably DML scaffolding, and is never presented as a faithful reconstruction.

Module scripting uses token-level head rewriting so create/alter/create-or-alter changes do not replace keywords inside comments, strings, or bodies. Table scripting synthesizes columns, types, defaults/computed expressions, identity, and constraints only when corresponding metadata is ready. DML emitters produce honest templates. The writer records semantic anchors so definition navigation can land on the object, parameter, column, or statement rather than merely opening line zero.

9.2 Strict freshness flow

The concrete scripting host calls the metadata lease’s strict policy before it pins. A host-side timeout is a backstop around the metadata policy timeout. Rejection, timeout, or an unavailable strict verdict refuses the operation; explicit offline-snapshot policy can instead allow cached output with provenance and an honest banner. This makes scripting stricter than interactive completion because generated DDL is more likely to be copied and executed.

Table 7: Definition and scripting authority modes.

Path	Authority check	Honest outcome
Local definition	Document analysis and overlay only	Return an in-document range; no metadata lease required.
Catalog definition / peek	Pinned object plus lazy module-definition read	Virtual content when available; encrypted, permission, unsupported, or not-loaded reason otherwise.
User-facing script generation	Host resolves strict freshness, then pins	F2/F1/F0 script with provenance, or explicit refusal when strict authority is unavailable.
Explicit offline scripting	Host selects <code>offlineSnapshot</code> policy	Cached output is allowed only with source/freshness/capture provenance and an offline banner.

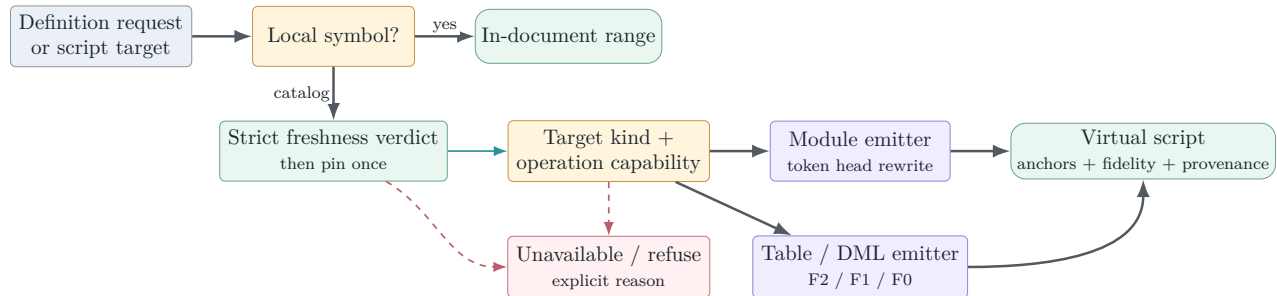


Figure 10: Definition and scripting truth flow. Local navigation stays in the document; catalog navigation produces a provenance-bearing virtual script or an explicit refusal.

10 Observability and privacy

The engine emits bounded spans for lexing, segmentation, parsing/sketching, completion, hover, signature help, diagnostics, definition, routing, and scripting. `perfTelemetry.ts` classifies `sqlLanguage.*` and `sqlScripting.*` under the existing `sqlLanguage` feature family; the accepted observability registry already covers those prefixes.

Normal diagnostics use safe scalar measures such as duration, counts, route, readiness, cache/hydration state, suppression reasons, and result shape. Rich completion diagnostics can classify prefix/name-part/top-label values as `user.text` or `object.name`; the shared diagnostics layer digests these by default and exposes them only under the explicitly elevated, time-bounded capture policy. The privacy claim is therefore not “identifiers never enter an event object.” It is that sensitive fields are classified at the call site and governed by the accepted capture/redaction boundary.

Observability contract

Do not add raw document text, SQL batches, connection strings, result rows, or unclassified object names to a language span. New attributes must be registered and typed before emission. Performance evidence should use counts, durations, readiness and suppression state; diagnosis that truly needs content must remain an explicit elevated-capture operation.

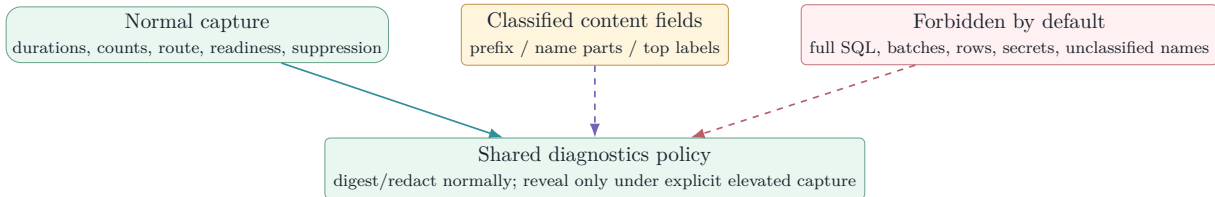


Figure 11: Observability boundary. The implementation is privacy-classified rather than content-blind: selected identifier fields may enter spans, but the shared capture policy normally digests them and forbids unclassified SQL or secrets.

11 Test design and validation

11.1 Focused semantic corpus

The 15 owned language and scripting unit files execute 707 tests at the pinned head: 707 passed, zero failed, zero skipped. The value is not just volume; the suites target incomplete syntax, partial truth, deterministic ordering, stale publication, refusal behavior, and every material repair through b66db2d8f. The exact-head sign-off also ran 47 adjacent scripting-service and observability tests, for 754 focused passes in total.

Table 8: Focused language and scripting inventory.

Area	Tests	Principal invariants
Completion	82	Member access, table sources, joins/FKs, expressions, CTE/derived/temp objects, DML/EXEC/DECLARE/USE, DDL declarations, star expansion, case-aware dedupe/edit ranges, partial-readiness honesty, latency budget, statement starts, static system catalog.
Cursor expectation	17	Category/confidence selection and quiet positions before completion producers run.
Core / router	24	Lexer coverage/Unicode whitespace, segmenter, feature routing, async timeout, maturity, fallback/circuit behavior, and LS-0 surface.
Definition	56	Aliases, variables, CTE/temp/script-local objects, catalog table/module scripts, honesty ladder, and capability routing.
Diagnostics	181	Lexical, GO, structural and recovery T1; 208/207/209 T2; expanded zero-unexpected-diagnostic honesty corpus; overlay/case/cache suppression; lazy hydration; engine pass.
Diagnostics freshness	6	Freshness gate and scheduler drift classification.

Area	Tests	Principal invariants
Hover	124	Catalog and local symbol families, H7 descriptions, partial truth, USE /cross-database/MERGE, DML positions, never-hover contexts, and system catalog.
Journal	3	Normal versus elevated-capture diagnostic fields.
Providers	16	Fourslash fixture parsing, fixture behavior, real CatalogBuilder /snapshot mapping, large filtered-prefix search, and unknown-case/offline honesty.
Scheduler	9	Debounce, sliced execution, stale document cancellation, metadata drift, and asynchronous factory recheck.
Signature	60	Built-ins, nesting/grouping, catalog routines, EXEC , and honesty positions.
Sketch	25	Total structural modeling across supported statement families, multipart names, module overlays, modern expression grammar, and incomplete text.
System catalog	16	Exact TVF result shapes, static data helpers, live-over-fallback precedence, and diagnostic interaction.
SQL scripting	77	Module rewrites/anchors/honesty; table F2/F1/anchors; comment-injection safety; DML; capabilities/routing; provenance/offline banners.
Strict scripting host	11	Freshness policy, timeout/refusal, pin ordering, provenance, and explicit offline behavior.
Total	707	Zero failures and zero skips in the owned focused selection.

11.2 Exact-head local validation and observed CI

Validation was rerun after review commit **b66db2d8f** on the topic branch rooted at extraction base **95cf790e5**:

Table 9: Independent local validation at **b66db2d8f**; hosted conclusions refreshed on 31 August 2026.

Lane	Result	Interpretation / exclusion
MSSQL build	Pass	Toolkit plus MSSQL extension typecheck, emit, extension/webview/notebook bundles.
MSSQL lint / repository format	Pass	Target ESLint and full-tree Prettier check completed without errors.
Owned language/scripting	707 pass; 0 fail	All files in table 8 ; no skipped cases.
Adjacent service/observability	47 pass; 0 fail	Scripting service and three accepted observability suites; 754 total focused passes.
Broad MSSQL hosted tests	4,742 pass; 0 fail; 10 skip	Inverse selection excludes only two independently documented current-main Windows baseline suites: Utility Tests - Path handling and QueryCompletionSoundService .
SQL Projects hosted tests	205 pass; 0 fail; 6 skip	Executed in its separate VS Code host.
Pre-commit / diff audit	Pass	Staged hooks and git diff -check clean; validation-generated toolkit locales restored; no manifest, setting, command, or lockfile change.
Production online package	Pass	Released STS 6.0.20260827.1 ; local artifact 1,542 files / 104.9 MB. Metafile contains no new language/scripting bundle input.

11.3 Review-machine scale and differential probes

The Fable review added useful **Reported evidence** evidence beyond the fixture suites. At pre-fix head `eafbbdc046`, it ran the then-current language/scripting and observability suites, hydrated the real `PR04 MetadataStore` with a 10,201-object B16 fixture, adapted it through `CatalogLanguageMetadataProvider`, and exercised `NativeSqlLanguageEngine`. It also ran malformed-input/depth probes and compared the curated system catalog with a live SQL Server 2025 instance. These observations motivated the repair corpus; exact-head local validation then reran the product suites at `b66db2d8f`.

Table 10: Pre-fix Fable review-machine measurements at `eafbbdc046`; observations, not product SLAs.

Probe	Observed	Interpretation
FROM completion, 10,201 objects	p50 4.0 ms; p95 8.7 ms	Current full-scan/sort cost is usable on the probe, but remains linear and uncached for empty-prefix catalog requests.
Member access / alias columns / hover	p50 about 0.1 ms; p95 ≤ 1 ms	Pinned lookup and local binding paths are fast in the fixture.
Completion at end of 2,000 statements	p50 0.8 ms; p95 35.6 ms	Tail latency shows why production size-tier budgets are still needed.
Diagnostics, 2,000 statements	68.5 ms cold; near 0 cached	Analysis reuse is effective when its cache key is sound.
Document symbols / folding, 2,000 lines	0.3 / 1.0 ms	Pure structural features are inexpensive on the probe.
Malformed/depth sweep	No throws; 5,000 parens in 13 ms	Supports total/tolerant direction; the separately found opener-at-EOF coverage hole is repaired and regression-tested.

The same live-catalog comparison found that all 157 curated built-in function names were genuine and identified two incorrect TVF result shapes. Review commit `b66db2d8f` removes the input handle names from `dm_exec_sql_text` / `dm_exec_query_plan`, pins their exact outputs, and adds a real `CatalogBuilder`/snapshot/provider-to-engine suite rather than relying only on fixture providers or hand-built handles.

11.4 GitHub validation snapshot

At report freeze, GitHub reports the PR open, mergeable, non-draft, labeled **Automated Review**, and still requiring human review. At head `b66db2d8f`, build-and-test, actions and JavaScript/TypeScript analysis, CodeQL, normalization, CLA, package creation, Azure VSCode MSSQL - Pull Request validation, and all six Linux/macOS/Windows x64/arm64 VSIX-launch jobs are green.

The GitHub package comparison is stronger than the absolute local size for this scope question: target and head are both 85,347 KiB for MSSQL, with zero delta for every packaged extension. Codecov reports 94.34331% patch coverage with 1,041 changed lines uncovered and project coverage rising from 78.22% to 79.86%. The largest uncovered concentration is the production metadata adapter; hover, diagnostics, definition, completion, sketch/context, cursor expectation, engine, and text snapshot also retain uncovered branches.

Evidence boundary

High patch coverage and 754 exact-head focused passes support the implementation, but the important properties are semantic: one generation per answer, suppression when metadata authority is unavailable,

no stale diagnostic publication, deterministic ranking, and strict scripting refusal. Coverage percentage alone cannot prove these; the differential probes were valuable precisely because they found ordinary constructs missing from the prior honesty corpus. Conversely, zero bundle-size delta proves inert packaging, not live feature quality.

Table 11: Largest reported pre-fix patch-coverage concentrations.

File / area	Patch age	cover-	Missing lines	Why it matters
provider/catalogProvider.ts	56.79%		178	The only production bridge to the merged metadata substrate; exact-head validation now adds one real builder/snapshot integration path beyond this earlier coverage report.
features/hover.ts	89.09%		115	Large name-chain and truth-tier surface with many context combinations.
features/diagnostics.ts	94.71%		109	High percentage, but uncovered branches can still be exactly the rare legal syntax that produces false squiggles.
features/definition.ts	88.06%		97	Crosses local-symbol resolution, scripting, async definition reads, and refusal states.
features/completion.ts	92.22%		90	Result edits, overlay routing, case rules, and partial-readiness paths remain semantically sensitive.
core/text/textSnapshot.ts	79.03%		26	Small file, but every range, anchor, and editor adaptation depends on coordinate correctness.

12 Known limitations and review closure

12.1 Finding distribution

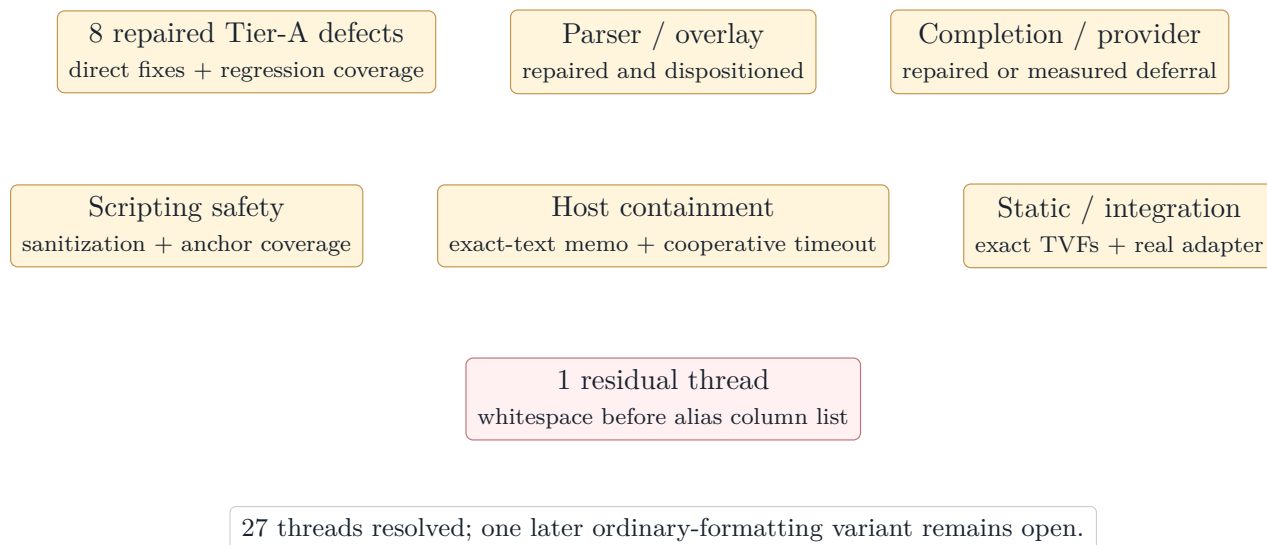


Figure 12: Review disposition map at `b66db2d8f`. The remaining red item is a narrow but valid semantic-trust repair.

12.2 Review snapshot: original closeout plus one residual

The pre-fix PR had 26 current review threads at `eafbbdc046`: two from Copilot and 24 from a later source-attributed Fable review posted through the PR author’s account. Every initial thread received an explicit disposition and was resolved after review commit `b66db2d8f`. One immediate Copilot follow-up about the documented newline precondition of `withHeader` was dispositioned after a complete production call-site audit, producing 27 resolved threads. A later Fable probe then found one valid residual variant: whitespace between a derived alias and its column list prevents the current raw-token-index guard from seeing the opening parenthesis. At report freeze that new thread is the sole unresolved thread.

The two original Copilot threads were narrow and are fixed:

1. `sqlScripting/scriptWriter.ts` now counts standalone CR, LF, and CRLF correctly in writer and prepended-header anchors, with mixed-ending regression coverage.
2. `sqlScripting/scriptingService.ts` now says synonym definition text is unavailable through this scripting engine rather than making an inaccurate hydration claim.

12.3 Tier A: ordinary-SQL trust defects

The eight original Tier-A findings were not architectural failures, but they were the exact user-trust failure class the diagnostic honesty design is intended to prevent. All eight have direct repairs at `b66db2d8f`; the fifth repair needs one additional significant-token walk to cover ordinary whitespace before the alias column list.

A1. Trailing comments and unterminated constructs (`nativeEngine.ts`). Lexical suppression now runs before the undefined-statement fallback; comment, string, SQLCMD, and exact-EOF opener cases return no completion.

A2. Alias assignment and positioned update (`diagnostics.ts`). Select-list `alias = expr` heads

and `CURRENT OF` cursor names are excluded from column diagnostics.

- A3. WITHIN GROUP (ORDER BY ...)** (diagnostics / keyword data). Contextual `WITHIN` plus ordered-set clause handling prevents the false missing-BY recovery.
- A4. AT TIME ZONE** (diagnostics / keyword data). Contextual `AT/ZONE` entries break the false expression-unit run without reserving legal identifiers.
- A5. Derived/VALUES alias lists** (`sketch/index.ts`). Unspaced `v(a,b)` lists are balance-skipped so commas cannot create phantom sources. **Open finding** A space or newline before `(a,b)` still leaves the guard on trivia; advance to the next significant token and add spaced `sketch/honesty` cases.
- A6. Non-breaking space in pasted SQL** (`lexer.ts`). Unicode horizontal space separators, including `U+00A0`, are whitespace rather than identifier characters.
- A7. UPDATE TOP (n) target** (`sketch/index.ts`). Update sketching skips `TOP` count forms and optional `PERCENT` before reading the target.
- A8. Default-schema shorthand `db..table`** (`sketch/index.ts`). Empty multipart components are preserved, and binder arity uses raw parts to reach honest cross-database/linked-server handling.

12.4 Tier B/C: correctness, safety, and evidence hardening

The other 16 Fable threads were triaged in five groups. Material correctness and safety issues were fixed; two design/optimization suggestions received narrower evidence-backed dispositions:

Table 12: Source-attributed review findings and final dispositions.

Group	Findings	Disposition
Parser / overlay honesty	<code>IS NOT DISTINCT FROM</code> ; labels/synonyms; script-created modules and qualified objects; EOF coverage; GO corpus.	Repaired with grammar, overlay, lexical-coverage, and table-driven regression tests.
Completion / provider	Temp DML targets; case-aware dedupe; bare- <code>alias</code> . edits; filtered prefix cap; unknown case rule; empty-prefix scan.	Correctness paths repaired. Prefix search grows past 1,200 filtered-out items. Empty-prefix caching is deferred: measured p95 is 8.7 ms and cache invalidation complexity is not justified without a budget regression.
Host correctness	Same-version/same-length cache collision; synchronous router timeout semantics.	Analysis and diagnostic memos require exact text. Async timeout is tested and cooperative semantics documented; synchronous interactive work remains self-bounding and diagnostic publication remains sliced.
Scripting safety	Comment placeholders/refusals can interpolate pathological identifiers or type text.	Reused <code>sanitizeCommentText</code> across DML and refusal paths; adversarial comment-terminator tests pass.
Static data / integration evidence	Two <code>dm_exec_*</code> TVF result schemas; fixture-only production adapter coverage.	Corrected exact TVF shapes and added real builder/snapshot/provider-to-engine coverage.

The original findings are resolved or explicitly dispositioned in the PR. The later spaced-alias-list reproduction remains a valid correctness repair. The fully green build/test matrix proves regression stability against the existing corpus; it does not negate a newly reproduced case outside that corpus.

12.5 Deliberate implementation limits

- The sketch is not a full T-SQL AST and the feature set is not a ScriptDOM equivalence claim.
- Highlights and semantic tokens are off.
- The bridge/fallback adapter is a router seam, not an implementation in this PR.
- The metadata adapter's lazy kick currently refreshes the full metadata ladder; section-specific hydration remains a future optimization.
- The system fallback is names-only and deliberately cannot support type/detail/key/definition claims.
- No live editor consumer, settings hierarchy, provider registration, Query Studio integration, AI completion, SQL document service, sqlcmd preprocessing, execution batch splitter, or query execution helper ships here.
- Unit fixtures cover a wide semantic corpus and the review added one SQL Server 2025/static-data plus 10,201-object store probe, but broad real-server collation, permissions, dynamic SQL, uncommon grammar, Azure/serverless behavior, and remote extension-host performance remain preview validation work.

12.6 Branch-drift boundary

The topic includes extraction base `95cf790e5`; the live PR API identifies base `53b707bbe`; and current repository `main` is `c6d2baa442`. The exact topic is three commits ahead of and nine commits behind current `main`. The mainline-only commits touch controller, document-service, package, test, and extension infrastructure. The PR remains mergeable and all hosted checks are green; every subsequent consumer must nevertheless start from refreshed `main` and repeat build, focused/broad semantics, package, launch, and review checks.

13 Integration and graduation plan

13.1 Next implementation step

The next PR should be a gated consumer, not another implicit activation edit scattered across the extension. Before that consumer is made reachable, the remaining spaced alias-list variant in [section 12.3](#) should be repaired and pinned by the honesty/suppression corpus. The consumer PR should then:

1. add one composition root that adapts VS Code documents and result types to the consumer-neutral contracts;
2. require `mssql.enableExperimentalFeatures` and a dedicated default-off native-language/AI feature flag before registration;
3. hide contributed commands and UI with complete manifest `when` clauses, not only disabled command enablement;
4. choose native versus existing behavior through the router, provide the real lazy bridge, and preserve per-feature fallback;

5. wire the metadata provider from the accepted shared catalog service and dispose leases/change subscriptions correctly;
6. carry perf test and scenario tests for activation-off invisibility, gated activation, latency, fallback, and stale-result suppression; and
7. avoid coupling AI completion to a global language-provider cutover—AI can be an early gated consumer of the same analysis/provider seam.

13.2 Graduation gates

Before moving from private preview toward default-on, validation should add:

1. a live-SQL truth corpus across supported versions, Azure variants, collations, permissions, synonyms, temporary/script objects, and cross-database contexts;
2. side-by-side native/STS result comparison with categorized differences rather than raw count parity;
3. interactive p50/p95/p99 budgets for completion/hover/signature and sliced whole-document diagnostics across document/catalog size tiers;
4. remote and web/virtualized extension-host CPU/memory measurements plus metadata hydration/retrigger behavior;
5. UI-level tests for flag hierarchy, command/menu invisibility, reload semantics, circuit-breaker fallback, and disabling the preview; and
6. an explicit privacy audit of elevated rich fields, journal/export behavior, and any AI consumer payload boundary.

Table 13: Minimum gates before the first user-reachable preview consumer.

Gate	Current state	Required closure
Ordinary-SQL trust corpus	Nearly closed	Original eight cases are repaired; add significant-token handling and spaced/newline alias-list regressions for the remaining variant.
Production metadata adapter	Partial	One real <code>CatalogBuilder</code> /snapshot/provider/engine path now exists; extend it with generation drift, partial readiness, and live-server variants.
Router and long-work containment	Partial	Explicit synchronous limitation, bounded algorithms, timeout test for yielding paths, and cooperative/worker plan for heavy work.
Private-preview composition	Not present	One default-off composition root with hierarchical flags, complete manifest visibility gates, disposal, and disable/reload tests.
Bridge/fallback	Interface only	Concrete lazy bridge, provider ownership, telemetry, and fallback behavior under throw/timeout/circuit.
Latency and memory	Probe only	p50/p95/p99 by document/catalog tier on local and remote extension hosts.
Privacy	Architecture present	Elevated-capture audit and product-level journal/export test once a consumer exists.

Table 14: Mechanism-to-evidence traceability.

Mechanism	Current evidence	Next proof
Pure analysis contracts	Import shape, fixture engines, hosted unit tests	Consumer adapter tests in the first activation PR.
Pinned-generation metadata	Provider types and fixture/null/system tests; reviewer real-store probe	CI end-to-end real-snapshot suite plus forced generation change.
No hot-path network wait	Synchronous pin contract, lazy kick tests, and reviewer real-store timings	Instrumented product/perftest assertion under hydration.
Completion determinism	Category/rank/ordinal tests; repaired suppression, dedupe, overlay, and edit-range cases	Close the spaced alias-list residual; add cross-platform replay.
Diagnostic honesty	T1/T2 split, expanded honesty suite, freshness/drift tests, and original Tier-A repairs	Pin the residual formatting variant and add a real-server/native-vs-STS differential corpus.
Stale-result suppression	Deterministic scheduler stamp tests	VS Code integration under edit/hydration storms.
Strict scripting provenance	Freshness/refusal/offline and emitter tests	Live permissions/encryption/definition corpus.
Failure containment	Three preference/maturity/throw-circuit/reset tests; sliced scheduler tests	Cooperative bound for synchronous work, timeout test, real bridge/fallback telemetry.
Private-preview invisibility	No manifest/controller/bundle inputs; zero package delta	Manifest/runtime gate tests when activation is introduced.

14 Final assessment

Key takeaway

PR #22860 is a strong extraction boundary. It converts the earlier language-service plan into a concrete, testable library without prematurely changing product behavior. The core architecture is the right one: tolerant analysis over unfinished text; a claim hierarchy that preserves lower-level truth; one pinned metadata generation per answer; explicit readiness and suppression; deterministic candidate order; sliced, stale-safe diagnostic publication; provenance-bearing scripting; and per-feature rollout/fallback.

The later review materially strengthened the correctness disposition. The eight original ordinary-SQL trust defects now have direct repairs and regression cases; the broader cache-identity, completion-edit, provider, scripting-sanitization, static-data, and real-adapter gaps are closed or explicitly dispositioned. The router timeout is accurately documented as cooperative rather than preemptive for synchronous work. One residual ordinary-formatting variant remains: whitespace before a derived alias column list can still create a phantom source and false 208 diagnostic. Green CI proves the encoded corpus is stable; zero package delta proves current users cannot reach the code. Neither makes that newly reproduced case disappear.

Decision statement

Recommended disposition at this snapshot: **accept the architecture and retain the source-only boundary, but close the one valid residual review thread before final automated sign-off.** Exact-head local and hosted validation is otherwise green, all prior threads are dispositioned, and the

PR is mergeable subject to human review. A later consumer PR must independently satisfy the full private-preview umbrella/feature gates, invisible command/menu contribution, runtime fail-closed behavior, and integration-level scenario validation before exposing the engine through Query Studio, a VS Code provider, command, or AI context path.

Evidence and source map

Reproducibility note

This report is pinned to an active PR. Review threads, Actions conclusions, main-branch refs, coverage, and mergeability are observations at the stated snapshot. Re-fetch the head and live PR state after any code or base-branch change before reusing its final disposition.

Evidence class	Primary sources used
PR state	PR #22860 , head <code>b66db2d8f</code> , extraction base <code>95cf790e5</code> , live API base <code>53b707bbe</code> , 27 resolved threads plus one later unresolved residual, Copilot review, source-attributed Fable review and probe report, exact-head sign-off comment, fully green Actions/Azure rollup, package-size bot, and Codecov report.
Public API / host	<code>extensions/mssql/src/sqlLanguage/api.ts</code> ; <code>host/nativeEngine.ts</code> ; <code>host/router.ts</code> ; <code>host/scheduler.ts</code> ; <code>host/scriptingHost.ts</code> .
Analysis core	<code>core/text/textSnapshot.ts</code> ; <code>core/lexer.ts</code> ; <code>core/segmenter.ts</code> ; <code>core/sketch/*</code> ; <code>core/parser/cursorExpectation.ts</code> ; <code>core/binder.ts</code> ; <code>core/overlay.ts</code> ; <code>database/name/fuzzy/quote</code> helpers.
Language features	<code>features/completion.ts</code> ; <code>diagnostics.ts</code> ; <code>hover.ts</code> ; <code>signatureHelp.ts</code> ; <code>definition.ts</code> ; <code>folding.ts</code> ; <code>documentSymbols.ts</code> ; generated keyword/built-in/default/system data.
Metadata seam	<code>provider/types.ts</code> ; <code>catalogProvider.ts</code> ; <code>systemCatalogView.ts</code> ; <code>nullProvider.ts</code> ; <code>fixtureProvider.ts</code> ; merged metadata-service implementation from PR #22836 .
SQL scripting	<code>extensions/mssql/src/sqlScripting/api.ts</code> ; <code>scriptingService.ts</code> ; <code>scriptWriter.ts</code> ; module/table/DML emitters; strict host under <code>sqlLanguage/host</code> .
Observability	<code>extensions/mssql/src/perf/perfTelemetry.ts</code> ; accepted diagnostics registry and privacy contracts already in <code>main</code> ; journal tests.
Tests	Fifteen <code>extensions/mssql/test/unit/sqlLanguage*.test.ts</code> and <code>sqlScripting*.test.ts</code> files; hosted JUnit XML; independent build/lint/broad-suite/package record summarized in table 9 .
Design intent	<code>vscode-ext-refactor/language-service-docs/05-tsql-language-service-design.md</code> ; <code>current_completions_parser_design.md</code> ; <code>PROGRESS.md</code> ; proposed diagnostics/completion designs. Code controls where historical/future design differs from the extraction.
Private-preview policy	Accepted PR04 preview-gating implementation plus the current PR manifest and activation diff; PR05 adds no consumer registration, setting, or command.